Universidade de Brasília
Facudade de Ciência e Tecnologia em Engenharias(FCTE)

Orientação por Objetos
FGA0158

Sistema de Vendas e Fidelidade
Cafeteria Geek "Byte & Brew"

Felipe Lima Santana (251039513)
Paulo Eduardo Fidelis Martins (242015325)
Pedro Magno Rabelo Teles (252039053)
João Vithor Camargo Emidio (251023264)
Eduardo Viana Ribeiro da Silva (251036440)

Prof. Andre Lanna

Brasília – DF
2026

1. Introdução

O objetivo deste trabalho é demonstrar na pratica as aplicações dos conhecimentos de Orientação a Objetos(OO), por meio do cenario Sistema de Vendas e Fidelidade Cafeteria Geek "Byte & Brew".Para organizar esse projeto, foi separado em 4 pacotes diferentes descritos abaixo:

modelo : Atedente, Cliente , Cliente_standard, Cliente_vip, Produto, Comida, Bebida, ItemPedido, Lista_cliente, Pedido, Produto.

servico : Promocao(interface), Venda, VerificaPontos, VerificaDuplicadeProduto, VerificadorCpf.

excecao : ClienteVipException, CpfInvalidoException, EstaNaListaExceptionProduto, EstoqueInsuficienteException, PontosINsuficientesException.

app : main

O UML está na ultima pagina desse documento.

2. Escolhas pessoais

A classe Atendente foi criada para começar a venda, ja que toda venda precisa de um atendente, sendo essa sua única funcionalidade, ela tem nome e CPF apenas.

Implementou-se a classe ListaCliente com o objetivo de amazenar os registros de todos os clientes cadastrados, a qual permite a busca por meio do CPF.

3. Herança Simples

Esse conceito foi aplicado a partir de duas superclasses principais: Cliente e Produto. A classe cliente deu origem a duas subclasses: ClienteStandard e ClienteVip.e a class produto deu duas classes filhas Comida e Bebida.

Os diagramas UML a seguir demonstram essa herança.

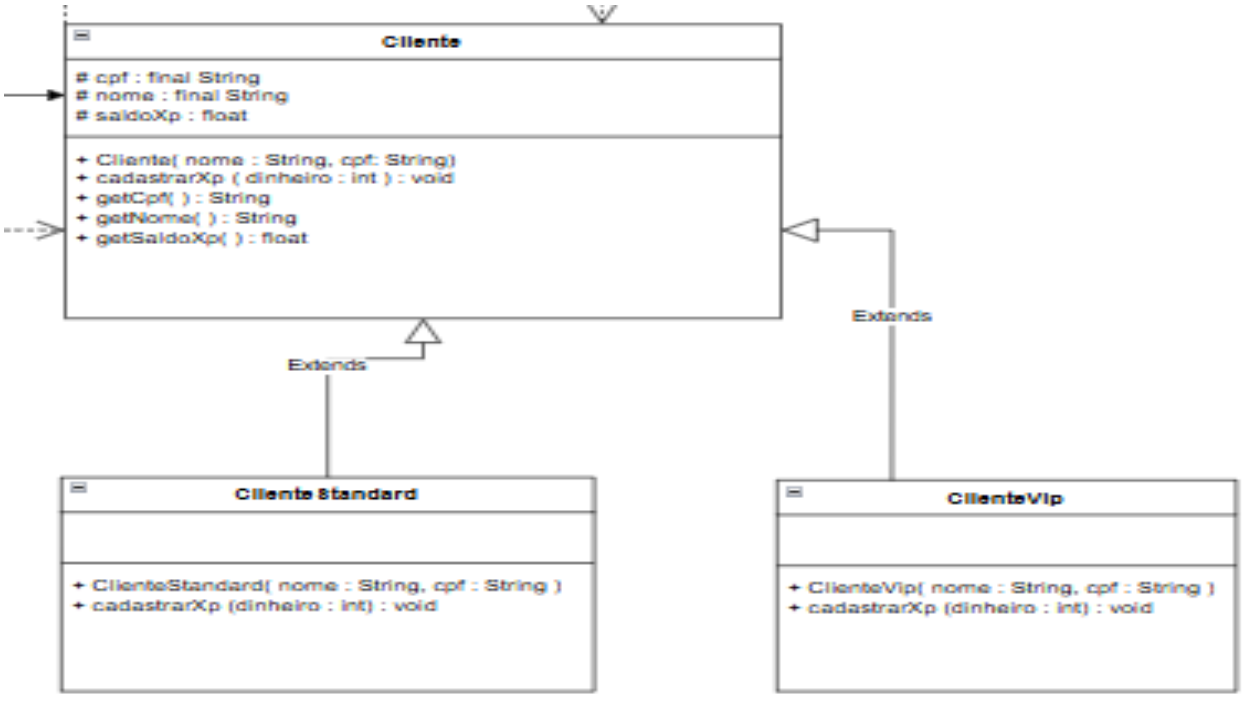

Imagem 1 – diagrama UML, Autoria própia

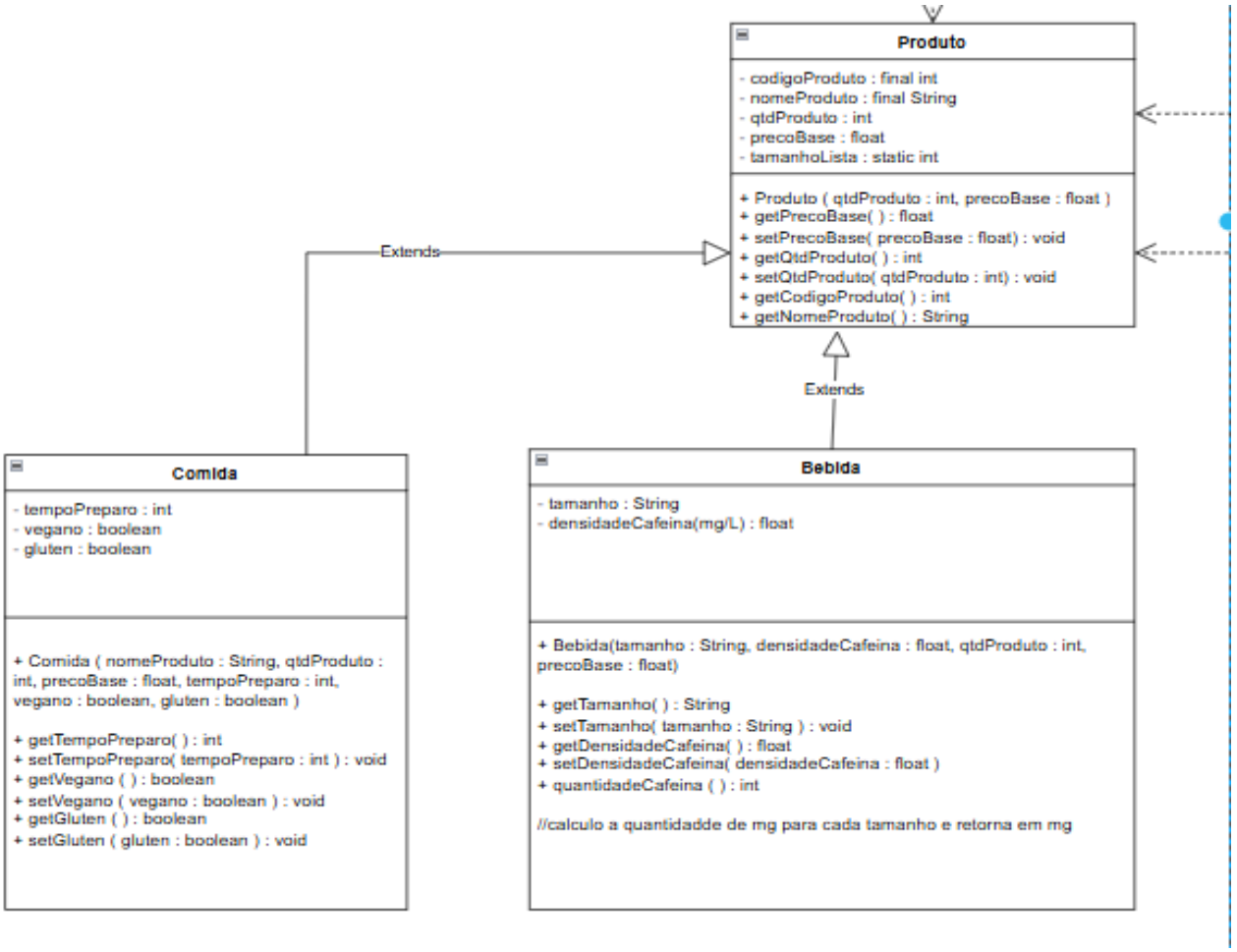

Imagem 2 – diagrama UML, Autoria própia

4. Polimorfismo por inclusão

O polimorfismo por inclusão foi empregado na classe ItemPedido, permitindo que uma única ArrayList do tipo Produto armazene tanto Comida quanto Bebida. Para garantir que a quantidade de produto não seja alterada no objeto original, criou-se um segundo ArrayList de inteiros, denominado quantidades; essa lista armazena a quantidade de produtos em um Pedido.

Como ambos os *arrays* trabalham em conjunto, a classe como um todo é considerada uma única lista.

Também se utilizou desse polimorfismo na classe ListaCliente, a qual possui um ArrayList de Cliente, armazenando assim todos os clientes cadastrados em um só lugar.

5. Polimorfismo por Sobrescrita

. O polimorfismo por sobrescrita está presente nas classes ClienteVip e ClienteStandard, manifestando-se no método cadastrarXp. A distinção entre as implementações reside na constante *n*, que multiplica o valor final da venda, cujo resultado é convertido/arredondado de float para inteiro.A constante multiplicadora n assume os sequintes valores dependedo da class: 0 em Cliente,1 em ClienteStandard, 2 em ClienteVip

6. Polimorfismo por Sobrecarga

Foi usado na classe ItemPedido os métodos adicionarProduto(Produto,int) e adicionar(Produto) a diferença entre ambos é que um adiciona apenas uma unidade do produto à lista, enquanto o outro permite definir a quantidade desejada.

Também foi usado os construtores na classe Pedido com Pedido() e Pedido(ItemPedido). O segundo construtor foi criado com uma lista de produtos já pronta. Isso foi feito para dar mais rapidez no debbuging.

Por causa disso, temos dois construtores na classe Venda, sendo o primeiro para o caso de cliente sem cadastro, no qual a instância de cliente é definida como null, e o segundo no qual é declarado qual cliente está fazendo a compra.

7. Interface

Isso foi obtido por meio da interface Promocao, a qual foi implementada na classe Venda, com os seguintes métodos: descontoDiaGeek e descontoPersonalizado.

8. Polimorfismo por Coerção

Utilizado para converter float para int, na aplicação de pontos Xp, demonstrando na seção 5 do relatório.

9. Classes, Objetos e Associações

Esse é o topico que mais foi utilizado nesse projeto, com diversas Associações, nesse documento focarei apenas nas interações que têm mais importante, para mais detalhes olha o UML em anexo.

A classe ItemPedido interage com Produto para colocar no ArrayList, a qual depois é associado à classe Pedido para amazenar a escolha do cliente.A classe de Venda associada a produto pode ou não depender de Cliente, já que cliente cadastrado é opcional, sendo sua unica associação obrigatoria sendo o Atendente.

No caso os verificadores é o que mais se utiliza no projeto, como o VerificadorCpf, uma classe que verifica se a string CPF é válido, e retorna um CPF formatado, também é usado para lança o erro CpfInvalidoException, por causa dissos, qualquer classe que tem CPF tem dependência direta com essa classe.

10. Modificadores de Escopo

Na classe Pedido, foi usado o modificador de escopo static para definir a variável tamanho_lista (não achamos um nome melhor). No construtor, o valor dela é atribuído à variável final comanda e, em seguida, incrementado em 1.

Também usamos a mesma lógica na classe Produto para gerar o codigoProduto, já que o único requisito era que o código fosse único para cada produto.

11. Exceções Customizadas

Para cada exceção, foi criada uma classe verificadora no pacote de serviços. Achamos que ficaria mais fácil no código demonstrar quando a exceção é gerada. Além disso, alguns verificadores também são utilizados para formatar a entrada de dados. Vale ressaltar que os verificadores não possuem construtores, todos contam com apenas um único método estático.

Como escolhemos armazenar o CPF como *string*, é necessário verificar se a entrada é um CPF válido. Assim, foi criada a exceção CpfInvalidoException, a qual é utilizada na classe, ele é lançado e lançada quando a *string* não possui 11 dígitos.

A ClienteVipException é utilizada na classe Venda. Quando tentamos adicionar pontos a um cliente que não é ClienteVip, ela lança o erro. Utilizando essa exceção, foi criado o método VerificaPontos, que primeiramente verifica se a venda possui um cliente cadastrado. Se houver cliente, ele verifica se o CPF pertence a uma instância de ClienteVip e, por último, aplica o desconto VIP caso ele tenha os pontos necessários; se não, lança a exceção PontosInsuficientesException..

A EstaNaListaExceptionProduto é lançada quando se tenta adicionar um produto ao ItemPedido que já está dentro do ArrayList. A verificação é feita na classe VerificaDuplicidadeProduto, que percorre o ArrayList. Se encontrar um produto com o mesmo código daquele que vai ser adicionado, ele lança um erro; caso contrário, retorna o produto.

Referências Bibliográficas

- LANNA, André. **fga0158**: trabalhoPratico. Versão: *master*. GitHub, 2026. Disponível em: https://github.com/andrelanna/fga0158/tree/master/trabalhoPratico. Acesso em: 5 jul. 2026.
- EMIDIO, João et al**.FGA0158-Trabalho_pratico**. Versão: *main*. GitHub, 2026. Disponível em: https://github.com/Joao-vithor-1/FGA0158-Trabalho_pratico/tree/main. Acesso em: 5 jul. 2026.

package servico

Package modelo

VerificaDuplicidadeProduto

+ verificaDuplicidadeProduto(lista : ItemPedido, produto : Produto) : Produto

VerificadorEstoque

+ verificadorEstoque(qtd_desejada : int, produto : Produto) : int

VerificadorCpf

+ static verificadorCpf(cpf : String) : String

<<Interface>>
Promocao

+ descontoDiaGeek (float : totalBebida) : float //aplica promoção em bebidas de 10%

+ descontoPersonalizado (float : totalBebida, float : totalComida, float : desconto) : int

Venda

- atendente : Atendente
- cliente : Cliente
- pedido : Pedido

+ Venda(atendente : Atendente, pedido : Pedido)
+ Venda(atendente : Atendente, pedido : Pedido, cliente : Cliente)
+ descontoDiaGeek(float : totalBebida) : float
- descontoClienteVip(float : TotalPedido, int : pontosXp) : float
+ descontoPersonalizado (float : totalBebida, float : totalComida, float : desconto) : int
+ finalizarVenda( ) : void
+ finalizarCompra (saldo : int) : void

VerificarPontos

+ verificarPontos(cliente : Cliente, pontosnecessario : int) : void

Atendente

- cpf : String
- nome : String

+ Atendente(cpf : String, nome : String)
+ getCpf( ) : String
+ setCpf(cpf : String) : void
+ getNome( ) : String
+ setNome(nome : String) : void

ListaCliente

- lista : ArrayList<Cliente>

+ adicionarCliente(cliente : Cliente) : void
+ buscarCliente(cpf : String) : boolean
+ buscarClientePorCpf(cpf : String) : Cliente
+ existeCliente(cpf : String) : boolean
+ removerCliente(cpf : String) : boolean
+ removerCliente(cliente : Cliente) : boolean
+ atualizarCliente(cpfAntigo : String, novoCliente : Cliente) : boolean
+ listarClientes() : ArrayList<Cliente>
+ buscaClienteVip(cpf : String) : boolean

Pedido

- itemPedido : ItemPedido
- comanda : final int
- totalPedido : float
- totalBebida : float
- totalComida : float
- tamanho_lista : static int

+ Pedido()
+ Pedido(ItemPedido itempedido)
+ adicionarProduto(produto : Produto, qtd_desejada : int) : void
+ adicionarProduto(prduto: Produto) : void
+ getTotalPedido () :float
+ getTotalComida () : float
+ getTotalBebida () : float
+ finalizarPedido (Atendente atendente) : Venda
+ finalizarPedido(Atendente atedente, Cliente cliente) : Venda

ItemPedido

- lista : ArrayList<Produto>
- quantidades : ArrayList<Integer>

+ adicionarProduto(produto : Produto, quantidadeDesejada : int) : void
+ removerProduto(produto : Produto) : void
+ getCodigoProduto(i : int) : int
# somaComida() : float
# somaBebida() : float
- imprimirComida() : void
- imprimirBebida() : void
+ imprimirCardapio() : void
+ quantidadeProdutos() : int
+ limparLista() : void
# baixaEstoque() : void

Produto

- codigoProduto : final int
- nomeProduto : final String
- qtdProduto : int
- precoBase : float
- tamanhoLista : static int

+ Produto (nomeProduto : String, qtdProduto : int, precoBase : float)
+ getPrecoBase( ) : float
+ setPrecoBase(precoBase : float) : void
+ getQtdProduto( ) : int
+ setQtdProduto(qtdProduto : int) : void
+ getCodigoProduto( ) : int
+ getNomeProduto( ) : String
+ reduzirEstoque(quantidade : int) : void

Cliente

cpf : final String
nome : final String
saldoXp : float

+ Cliente(nome : String, cpf: String)
+ cadastrarXp (dinheiro : int) : void
+ getCpf( ) : String
+ getNome( ) : String
+ getSaldoXp( ) : float

ClienteStandard

+ ClienteStandard(nome : String, cpf : String)
+ cadastrarXp (dinheiro : int) : void

ClienteVip

+ ClienteVip(nome : String, cpf : String)
+ cadastrarXp (dinheiro : int) : void

Comida

- tempoPreparo : int
- vegano : boolean
- gluten : boolean

+ Comida (nomeProduto : String, qtdProduto : int, precoBase : float, tempoPreparo : int, vegano : boolean, gluten : boolean)

+ getTempoPreparo( ) : int
+ setTempoPreparo(tempoPreparo : int) : void
+ getVegano ( ) : boolean
+ setVegano (vegano : boolean) : void
+ getGluten ( ) : boolean
+ setGluten (gluten : boolean) : void

Bebida

- tamanho : String
- densidadeCafeina(mg/L) : float

+ Bebida(tamanho : String, densidadeCafeina : float, qtdProduto : int, precoBase : float)

+ getTamanho( ) : String
+ setTamanho(tamanho : String) : void
+ getDensidadeCafeina( ) : float
+ setDensidadeCafeina(densidadeCafeina : float)
+ quantidadeCafeina ( ) : int

//calculo a quantidadde de mg para cada tamanho e retorna em mg

Package excecao

EstoqueInsuficienteException

- nomeProduto: String
- quantidadeSolicitada: int
- quantidadeDisponivel: int

+ EstoqueInsuficienteException(
nomeProduto: String,
quantidadeSolicitada: int,
quantidadeDisponivel: int)
+ getNomeProduto(): String
+ getQuantidadeSolicitada(): int
+ getQuantidadeDisponivel(): int

<<built-in>>
Exception

PontosInsuficientesException

- pontosNecessarios : float
- pontosDisponiveis : float

+ PontosInsuficientesException(pontosNecessarios: float, pontosDisponiveis: float)
+ getPontosNecessarios() float
+ getPontosDisponiveis() float

EstaNaListaExcptionProduto

- produto : Produto
- lista : ItemPedido

- EstaNaListaExceptionProduto (produto : Produto, lista : ItemPedido, erro : String)

CpfInvalidoException

- cpf : String

+ CpfInvalidoException

ClienteVipException

- ClienteVipException : Cliente

+ ClienteVipException (Cliente client,String erro)

Use
Extends